

Exercise 7

Advanced Methods for Regression and Classification

Kyzer Gerez

26.11.2025

```
library(fastDummies)

# read csv and drop duration col
d <- subset(read.csv2("bank.csv"), select=-c(duration))

# check for na
rownames(d[rowSums(is.na(d)) > 0,])

## character(0)

# convert month to numeric representation
d["month"] <- match(d[["month"]], tolower(month.abb))

# convert binary variables to numeric representation
d$default <- ifelse(d$default=="no", 0, 1)
d$housing <- ifelse(d$housing=="no", 0, 1)
d$loan <- ifelse(d$loan=="no", 0, 1)
d$y <- ifelse(d$y=="no", 0, 1)

# split into 3/4 train, 1/4 test
smp_size <- floor(3/4 * nrow(d))

# set the seed to make partition reproducible
set.seed(1)
train_ind <- sample(seq_len(nrow(d)), size = smp_size)

train <- d[train_ind, ]
test <- d[-train_ind, ]
```

1. Use the same data set `bank.csv` as in the last exercise, also without variable “duration”.
 - (a) Use logistic regression for a training set (function `glm()` with `family="binomial"`). Look at the inference table (with `summary()`) and interpret the outcome.

```
library(MASS)

# train model
mod.log.reg <- glm(y ~., train, family="binomial")

summary(mod.log.reg)
```

```
##
## Call:
```

```
## glm(formula = y ~ ., family = "binomial", data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -1.797e+00  5.911e-01  -3.040 0.002368 **
## age            2.997e-03  7.093e-03   0.423 0.672658
## jobblue-collar -1.750e-01  2.430e-01  -0.720 0.471304
## jobentrepreneur -2.312e-01  3.902e-01  -0.592 0.553531
## jobhousemaid   2.901e-01  3.953e-01   0.734 0.462990
## jobmanagement  2.256e-02  2.418e-01   0.093 0.925660
## jobretired     9.443e-01  2.989e-01   3.160 0.001580 **
## jobself-employed 3.001e-01  3.286e-01   0.913 0.361164
## jobservices   -2.197e-01  2.821e-01  -0.779 0.436168
## jobstudent     4.024e-01  3.937e-01   1.022 0.306784
## jobtechnician  -2.022e-01  2.297e-01  -0.880 0.378881
## jobunemployed  -2.013e-02  3.852e-01  -0.052 0.958331
## jobunknown     7.622e-01  5.146e-01   1.481 0.138566
## maritalmarried -6.130e-01  1.699e-01  -3.608 0.000308 ***
## maritalsingle  -2.958e-01  2.015e-01  -1.468 0.141997
## educationsecondary 3.095e-01  2.023e-01   1.530 0.126030
## educationtertiary 4.812e-01  2.356e-01   2.043 0.041062 *
## educationunknown -2.756e-01  3.537e-01  -0.779 0.435914
## default        5.232e-01  4.017e-01   1.302 0.192799
## balance        -2.102e-05  2.336e-05  -0.900 0.368063
## housing        -2.542e-01  1.277e-01  -1.990 0.046562 *
## loan          -6.852e-01  2.016e-01  -3.399 0.000675 ***
## contacttelephone 1.066e-01  2.099e-01   0.508 0.611646
## contactunknown  -7.308e-01  1.736e-01  -4.209 2.57e-05 ***
## day           -1.107e-03  7.219e-03  -0.153 0.878109
## month          1.875e-02  2.287e-02   0.820 0.412344
## campaign       -6.694e-02  2.755e-02  -2.430 0.015113 *
## pdays          5.964e-04  1.010e-03   0.591 0.554786
## previous       1.625e-02  4.249e-02   0.382 0.702145
## poutcomeother   3.563e-01  2.729e-01   1.306 0.191669
## poutcomesuccess 2.348e+00  2.843e-01   8.258 < 2e-16 ***
## poutcomeunknown -1.647e-02  3.231e-01  -0.051 0.959338
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 2407.7  on 3389  degrees of freedom
## Residual deviance: 2114.6  on 3358  degrees of freedom
## AIC: 2178.6
##
## Number of Fisher Scoring iterations: 5
```

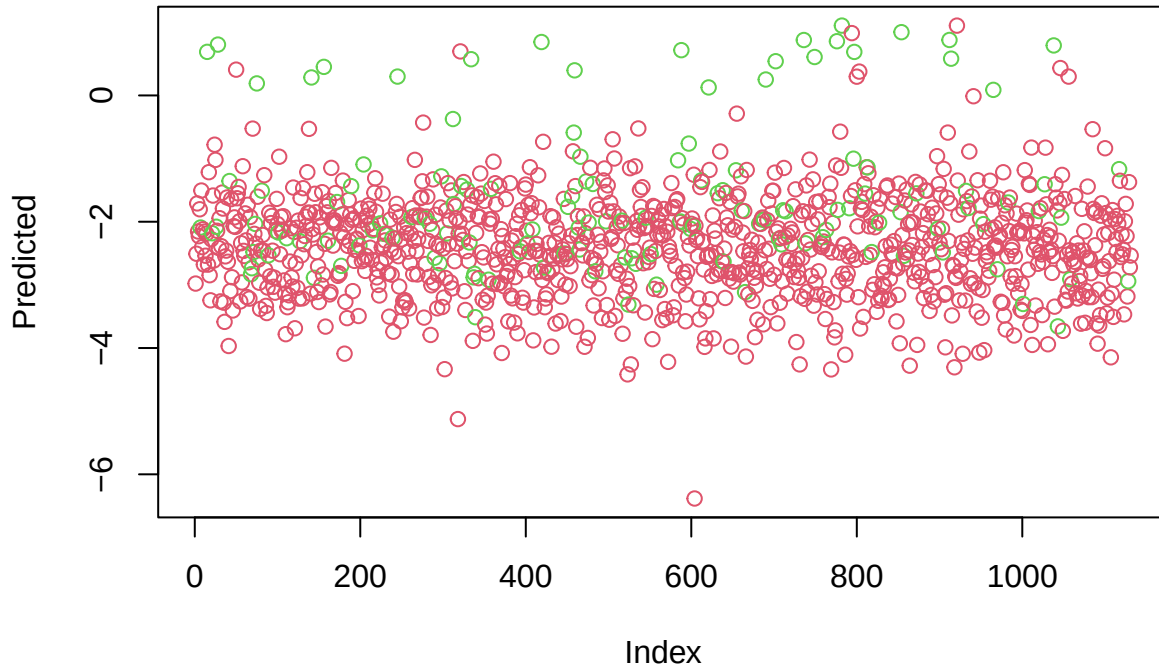
The summary shows that the predictors marital, loan, housing, and campaign are significant in predicting the response. Additionally, the values job-retired, education-tertiary, contact-unknown, and poutcome-success are significant.

- (b) Use the model to predict the group label of the remaining test set observations (what does the function actually predict by default?). Compute the misclassification rate for every group separately, show the confusion table, and report the balanced accuracy.

The misclassification rate for each group can be calculated as $\frac{FN+FP}{n}$ where n is the number of instances for the given group.

```
bank.test.pred <- predict(mod.log.reg, test)
```

```
plot(bank.test.pred,col= test$y+2,
     ylab="Predicted")
```



```
# convert from linear to binary scale
```

```
bank.test.pred.mod <- ifelse(bank.test.pred<=0, 0, 1)
```

```
# confusion matrix and balanced accuracy
```

```
library(caret)
```

```
bank.test.confusion <- confusionMatrix(data=as.factor(bank.test.pred.mod),
                                       reference=as.factor(test$y),
                                       positive = "1")
```

```
bank.test.confusion$table
```

```
##           Reference
## Prediction  0    1
##           0 989 111
##           1   8   23
```

```
bank.test.confusion$byClass["Balanced Accuracy"]
```

```
## Balanced Accuracy
```

```
##           0.5818089
```

```
# compute misclassification rate for no
```

```
no.total <- bank.test.confusion$table[1,1] + bank.test.confusion$table[2,1]
```

```
no.FN <- bank.test.confusion$table[2,1]
```

```
no.FP <- bank.test.confusion$table[1,2]
```

```
bank.misclass.no <- round((no.FN + no.FP) / no.total, 3)
```

```

# compute misclassification rate for yes
yes.total <- bank.test.confusion$table[1,2] + bank.test.confusion$table[2,2]
yes.FN <- bank.test.confusion$table[1,2]
yes.FP <- bank.test.confusion$table[2,1]
bank.misclass.yes <- round((yes.FN + yes.FP) / yes.total, 3)

# output misclassification rates
print(paste("The misclassification rate for 'no' is", bank.misclass.no))

## [1] "The misclassification rate for 'no' is 0.119"

print(paste("The misclassification rate for 'yes' is", bank.misclass.yes))

```

```
## [1] "The misclassification rate for 'yes' is 0.888"
```

The predictions are returned in the scale of the linear predictor, with zero as the decision boundary. The model predicts “no” by default since the intercept is a negative value.

The colors in the plot indicates the true class of the observation, with red being “no” and green being “yes”.

- (c) A way to consider the problem of imbalanced groups is to assign a weight to every observation, by using the weights argument in the `glm()` function. How do you have to select the weights, and what is the resulting confusion table and balanced accuracy?

The weights can simply be the proportion of each class for the train set. The weight for the minority class would be the proportion of the majority class and vice versa.

```

# calculate weights
weight.0 <- nrow(train[train$y == 1,]) / nrow(train)
weight.1 <- nrow(train[train$y == 0,]) / nrow(train)

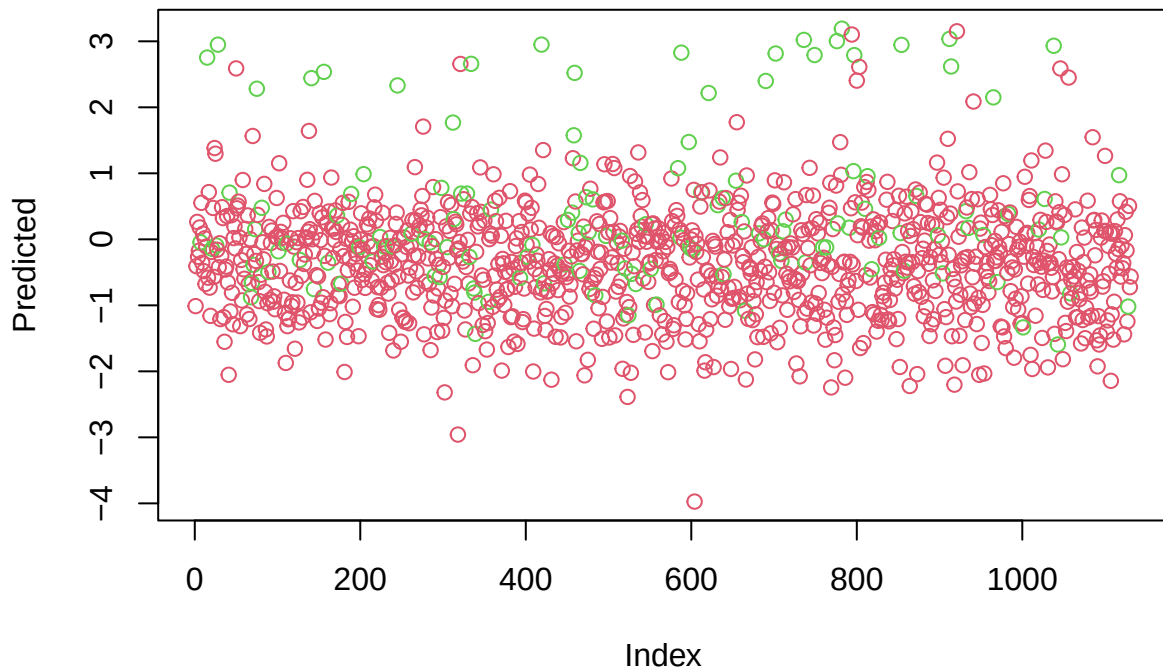
# create weight vector
weights <- ifelse(train$y == 0, weight.0, weight.1)

# train weighted model
mod.weighted <- glm(y ~ ., train,
                    family="binomial",
                    weights=weights)

# predict
bank.weighted.test.pred <- predict(mod.weighted, test)

plot(bank.weighted.test.pred, col= test$y+2,
     ylab="Predicted")

```



```
# convert from linear to binary scale
bank.weighted.test.pred.mod <- ifelse(bank.weighted.test.pred<=0, 0, 1)

# confusion matrix and balanced accuracy
bank.weighted.test.confusion <- confusionMatrix(data=as.factor(bank.weighted.test.pred.mod),
                                                reference=as.factor(test$y),
                                                positive = "1")

bank.weighted.test.confusion$table
```

```
##           Reference
## Prediction  0    1
##           0 672  57
##           1 325  77
```

```
bank.weighted.test.confusion$byClass["Balanced Accuracy"]
```

```
## Balanced Accuracy
##           0.6243245
```

```
# compute misclassification rate for no
no.total <- bank.weighted.test.confusion$table[1,1] +
  bank.weighted.test.confusion$table[2,1]
no.FN <- bank.weighted.test.confusion$table[2,1]
no.FP <- bank.weighted.test.confusion$table[1,2]
bank.misclass.no <- round((no.FN + no.FP) / no.total, 3)

# compute misclassification rate for yes
yes.total <- bank.weighted.test.confusion$table[1,2] +
  bank.weighted.test.confusion$table[2,2]
yes.FN <- bank.weighted.test.confusion$table[1,2]
yes.FP <- bank.weighted.test.confusion$table[2,1]
bank.misclass.yes <- round((yes.FN + yes.FP) / yes.total, 3)

# output misclassification rates
```

```
print(paste("The misclassification rate for 'no' is", bank.misclass.no))
```

```
## [1] "The misclassification rate for 'no' is 0.383"
```

```
print(paste("The misclassification rate for 'yes' is", bank.misclass.yes))
```

```
## [1] "The misclassification rate for 'yes' is 2.851"
```

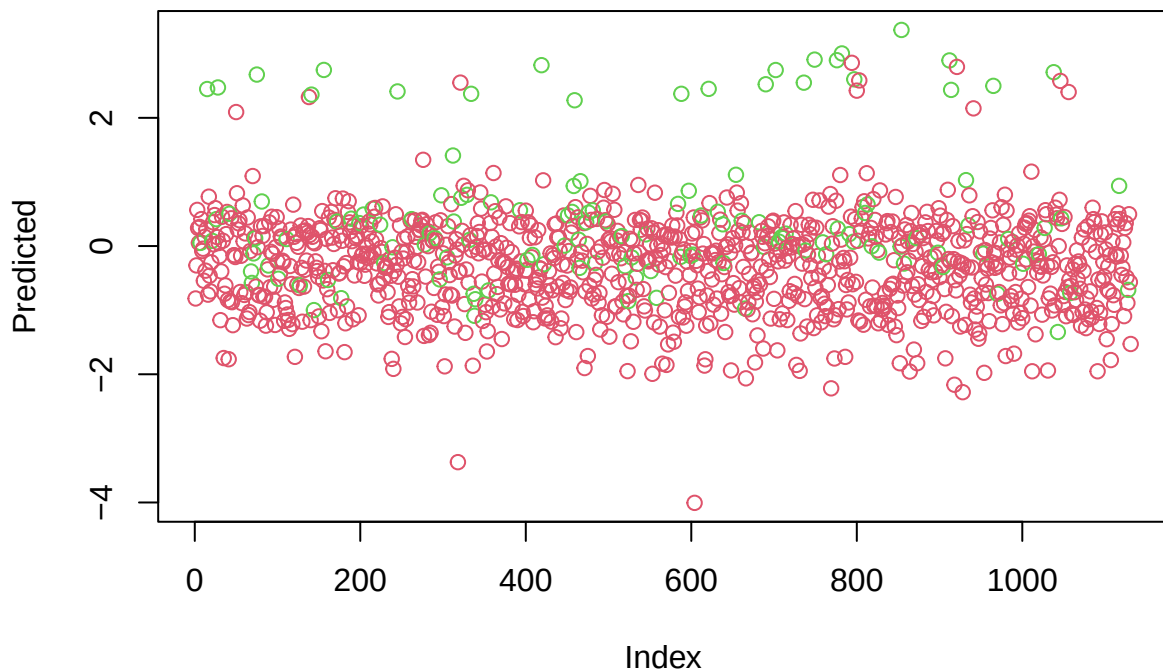
The weighted model has an improved balanced accuracy at the cost of a much higher misclassification rate for the 'yes' group.

- (d) Based on the model from 1(c), use stepwise variable selection with the function `step()` to simplify the model. Does this also lead to an improvement of the balanced accuracy?

```
# backward selection
mod.back <- step(mod.weighted,
                 direction = "backward",
                 trace=0)

# predict
bank.back.test.pred <- predict(mod.back, test)

plot(bank.back.test.pred, col= test$y+2,
     ylab="Predicted")
```



```
# convert from linear to binary scale
bank.back.test.pred.mod <- ifelse(bank.back.test.pred<=0, 0, 1)

# confusion matrix and balanced accuracy
bank.back.test.confusion <- confusionMatrix(data=as.factor(bank.back.test.pred.mod),
                                             reference=as.factor(test$y),
                                             positive = "1")

bank.back.test.confusion$table
```

```
##           Reference
```

```
## Prediction    0    1
##              0 648  49
##              1 349  85
```

```
bank.back.test.confusion$byClass["Balanced Accuracy"]
```

```
## Balanced Accuracy
##              0.6421391
```

The balanced accuracy for the backward selection model is 0.642. This is slightly better than the balanced accuracy for the original model, which is 0.624.

2. Use the data set `data(Khan)` from the package `ISLR`. The data set consists of a number of tissue samples corresponding to four distinct types of small round blue cell tumors. For each tissue sample, 2308 gene expression measurements are available (see also help file). The task is to train a classifier based on the training data (`Khan$xtrain`, `Khan$ytrain`), use it for predicting the class of the test data (`Khan$xtest`), and to evaluate the predictions using the group information of the test data (`Khan$ytest`).

```
library(ISLR)
data(Khan)
```

```
print(paste("n =", nrow(Khan$xtrain)))
```

```
## [1] "n = 63"
```

```
print(paste("p =", ncol(Khan$xtrain)))
```

```
## [1] "p = 2308"
```

- (a) Why would LDA or QDA not work here? Would RDA work (you can either try it out, or simply argue)?

LDA and QDA requires that the sample covariance matrix to be full rank and invertable, which does not happen if $n < p$. For the Khan dataset, $n < p$. In contrast, RDA works under this condition because it ensures stability of the covariance matrix by shifting it towards a diagonal matrix or pooled covariance matrix using the two tuning parameters.

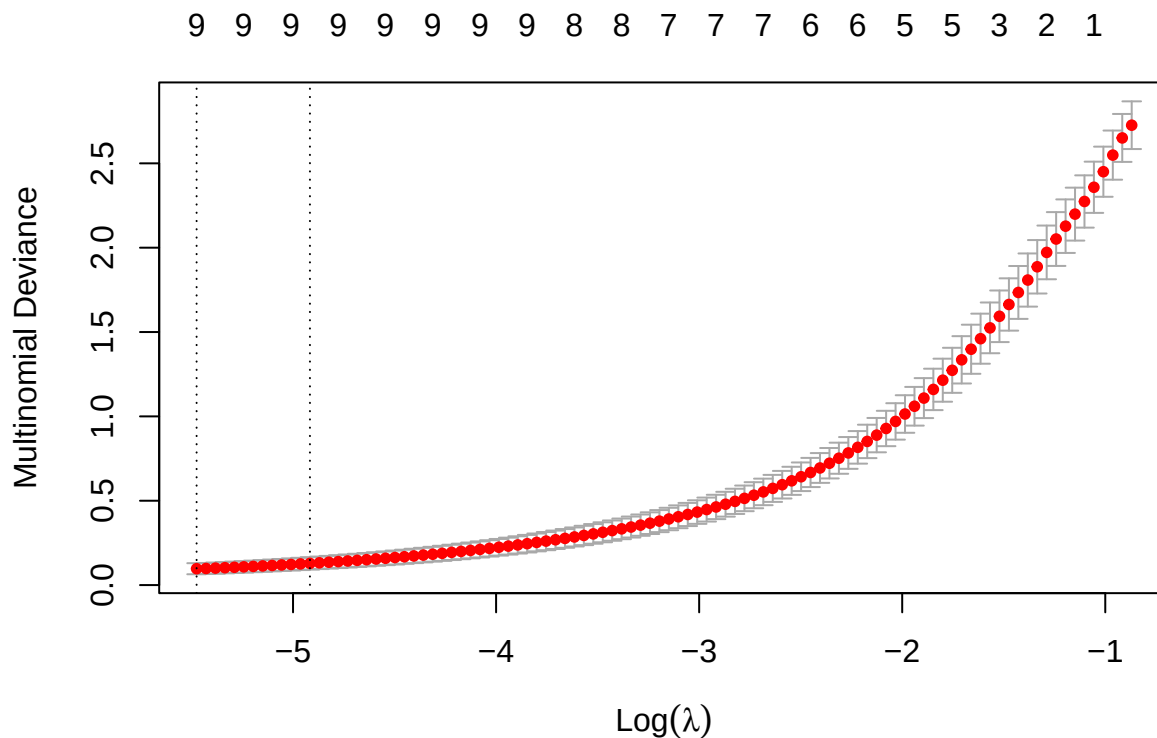
- (b) Use the function `cv.glmnet()` from the package `glmnet`, with the argument `family="multinomial"`, to build a model for the training set (the response might need to be converted to a factor). Plot the outcome object. What do you conclude? What is the objective function to be minimized?

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-10
```

```
mod.multi.glm <- cv.glmnet(x=Khan$xtrain, y=Khan$ytrain,
                          family="multinomial")
plot(mod.multi.glm, sign.lambda=1)
```



The plot shows that the best performing models have small lambda values, which means that coefficients are not shrunk significantly. However, the best models still contain only nine non-zero coefficients out of the original 2308 predictors.

The objective function to be minimized is the log-likelihood function, and the results are represented by the y-axis, multinomial deviance. The deviance is the negative contribution of each observation to the log-likelihood function.

- (c) Which variables contribute to the model? To see this, you can use `coef()` for the output object. You obtain an object with 4 (= number of groups) list elements, containing the estimated regression coefficients. Thus, this is different from our approach to logistic regression with K groups in the course notes, where you would only obtain $K - 1$ coefficient vectors.

```
multi.glm.betas <- coef(mod.multi.glm) # 1.se
multi.glm.beta.1 <- multi.glm.betas$`1`
multi.glm.beta.2 <- multi.glm.betas$`2`
multi.glm.beta.3 <- multi.glm.betas$`3`
multi.glm.beta.4 <- multi.glm.betas$`4`

print(paste("The coefficient vector for group 1 contains the variables",
  toString(multi.glm.beta.1[i[2:length(multi.glm.beta.1[i])]))))

## [1] "The coefficient vector for group 1 contains the variables 1, 123, 589, 836, 846, 1066, 1387"
print(paste("The coefficient vector for group 2 contains the variables",
  toString(multi.glm.beta.2[i[2:length(multi.glm.beta.2[i])]))))

## [1] "The coefficient vector for group 2 contains the variables 246, 545, 1319, 1389, 1954, 2050"
print(paste("The coefficient vector for group 3 contains the variables",
  toString(multi.glm.beta.3[i[2:length(multi.glm.beta.3[i])]))))

## [1] "The coefficient vector for group 3 contains the variables 255, 575, 695, 742, 842, 879, 176"
```

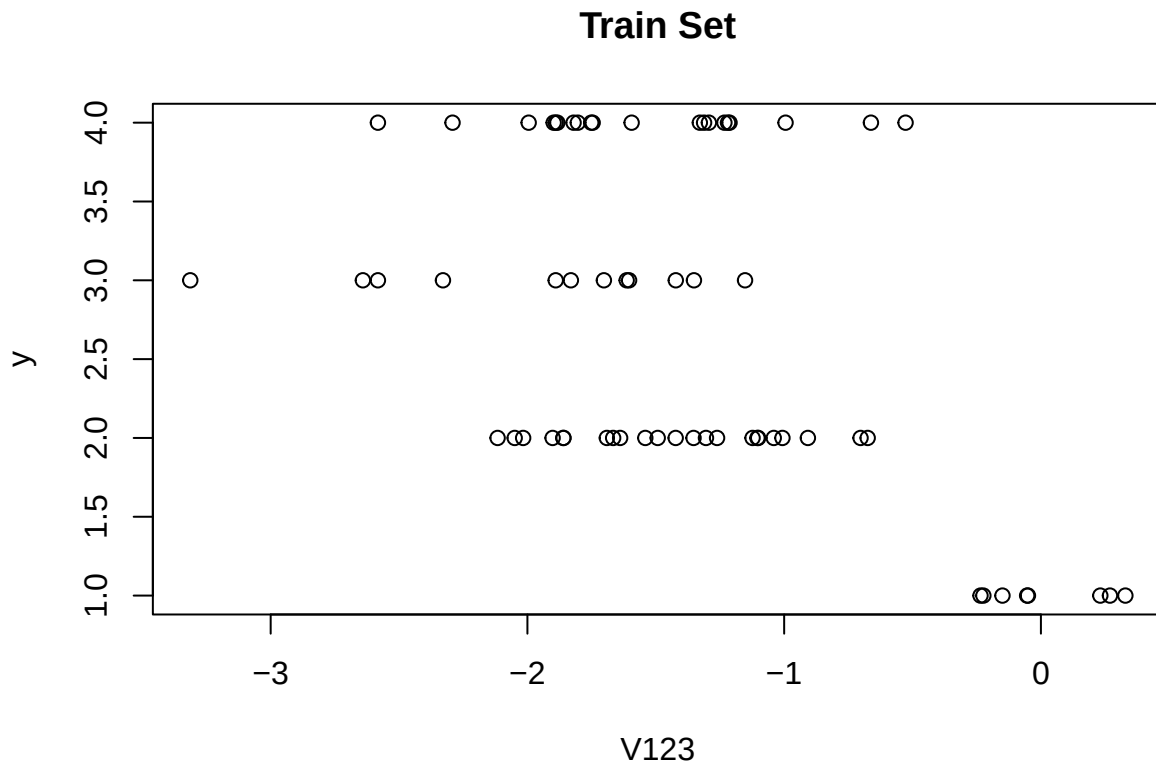


```
print(paste("The coefficient vector for group 4 contains the variables",
           toString(multi.glm.beta.4@i[2:length(multi.glm.beta.4@i)])))
```

```
## [1] "The coefficient vector for group 4 contains the variables 174, 509, 554, 910, 1003, 1055, 1
```

- (d) Select one of the variables from 2(c) which is relevant e.g. for the first group, and plot this variable against the response (using the training data). What you should see is that the values of the first group clearly differ from those of the other groups.

```
# V123 vs y
plot(Khan$xtrain[,123], Khan$ytrain,
     ylab="y",
     xlab="V123",
     main="Train Set")
```



- (e) Now use the trained model and predict the group membership of the test data. Be careful, `predict()` yields predictions for each observation to each class, and you need to select the appropriate class. Report the confusion table and the misclassification error for the test data.

```
Khan.test.pred <- predict(mod.multi.glm, Khan$xtest)

# iterate by row and column to determine predicted class for each obs
Khan.test.pred.mod <- numeric(length(Khan$ytest))
for(i in 1:nrow(Khan.test.pred)) {
  class <- 0
  max.prob <- -Inf
  for(j in 1:ncol(Khan.test.pred)) {
    if(Khan.test.pred[i, j] > max.prob) {
      class <- j
      max.prob <- Khan.test.pred[i, j]
    }
  }
}
```

```

    }
    Khan.test.pred.mod[i] = class
  }

# confusion table
Khan.confusion <- confusionMatrix(data=as.factor(Khan.test.pred.mod),
                                   reference=as.factor(Khan$ytest))

Khan.confusion$table

##           Reference
## Prediction 1 2 3 4
##           1 3 0 0 0
##           2 0 6 0 0
##           3 0 0 6 0
##           4 0 0 0 5

```

The confusion table shows that the model perfectly predicted each class i.e. the misclassification rate for each class is zero.